Introduction to supervised learning

classification

Dr. Zakarya Farou

Data Science and Engineering Department

zakaryafarou@inf.elte.hu

Plan

1. Introduction
2. Decision trees
3. Naive Bayes Classifiers
4. Comparison and summary
5. Bibliography

Distance-based approaches and validation criteria are covered in Lecture 7.2.

Learning objectives

By the end of this lecture you should be able to:

1. Explain the roles of the training, validation and test sets.
2. Describe the bias–variance trade-off and recognise overfitting.
3. Build a decision tree by hand using entropy and information gain.
4. Compute a Naive Bayes posterior, including Laplace smoothing.
5. Choose between a decision tree and Naive Bayes for a given problem.

Outline

1 Introduction

2 Decision trees

3 Naive Bayes Classifiers

4 Comparison and summary

5 Bibliography

What is supervised Learning ?

Supervised learning is a Machine Learning task the aim of which is to learn a model from **labeled training data** in order to predict the target of new, unlabeled data. It covers two families of tasks:

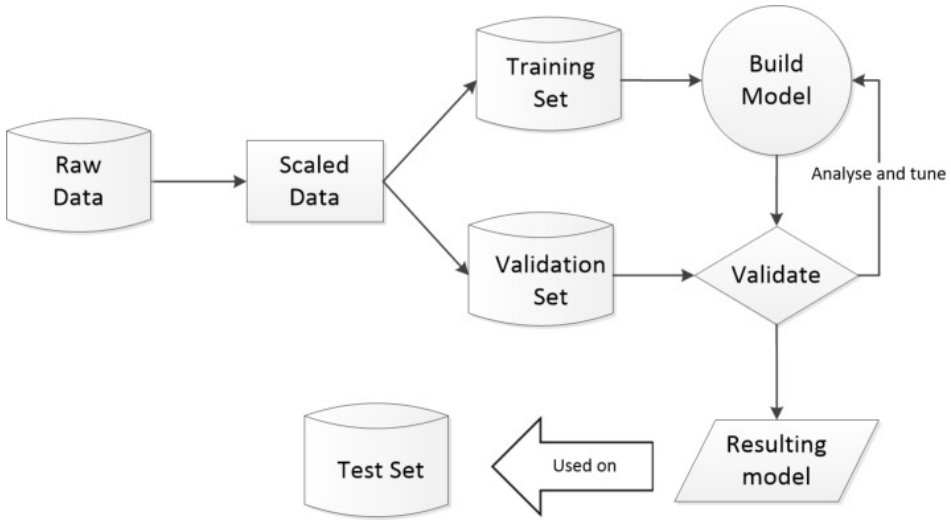
- **Classification**: the target is discrete (a class).
  - Speech and hand-writing recognition, spam filtering
  - Pattern Recognition, medical diagnosis
- **Regression**: the target is a real number.
  - Price prediction, demand forecasting

Supervised Learning workflow

In supervised learning, we distinguish 3 sets of labeled data:

- The **Training set**, on which the labels are known and that is used to build the model.
- The **Validation set**, on which the labels are also known and that is used to tune the model and compare models. (not used by all algorithms)
- The **Test set**: labeled data never seen during training or tuning, used *once* at the end to estimate real performance.

The final model is then applied to **new, unlabeled data**.



Avoid data leakage

Data leakage

Information from the validation or test set must never be used while building the model. Otherwise the measured performance is too optimistic.

- Fit the scaling parameters (mean, standard deviation, min, max) on the **training set only**, then apply them unchanged to the validation and test sets.
- The same holds for feature selection, imputation of missing values, etc.
- Look at the test set only once: tuning on it turns it into a second validation set.

Supervised Learning data sets examples

Supervised data are described by several attributes and a target class (or label), which is known in the training, validation and test sets, but unknown for new data.

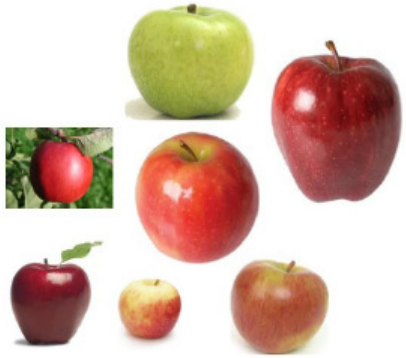
Size	Weight	Shoe size	Sex
176	72	43	M
159	61	37	F
180	66	39	F
185	85	44	M
177	70	41	F
155	88	38	M
210	110	45	M

Table 1: Training data

Size	Weight	Shoe size	Sex
205	85	47	?
172	60	40	?
164	57	38	?
169	52	36	?
183	78	42	?
175	65	44	?
191	77	41	?

Table 2: New (unlabeled) data

Supervised Learning Example



Labeled images of apples



Labeled images of peaches



Mixture of fruits images to classify

The classifier does not see "an apple": it sees features such as **colour**, **size** and **skin texture**, exactly like the attributes of the previous tables.

The notion of classifier

Apart from regression, most supervised algorithms are known as **classifiers**.

Classifiers

- A classifier learns a model in the form of a function, a set of logic rules, the parameters of a probabilistic model, the parameters of a neural network, a set of prototypes, etc.
- The classifier will use the model it learned to label new and previously unknown data.

- When the target attribute is **discrete**, the task is a **classification**; its possible values are called **classes** or **labels** (the two words are synonyms).
- When the target attribute is a **real number**, the task is a **regression**.

Formalism

- Let us denote $\mathbf{X} = \{x_1, \cdots, x_N\}, x_i \in \mathcal{X}$ the matrix of the N observed examples of the training set.
  - The x_i are vectors containing the attributes of each object.
- Let $Y = \{y_1, \cdots, y_N\}, y_i \in [1..K]$ be the vector containing the labels/classes associated to the observed examples.
- We note $\mathcal{L} = \{(x_i, y_i), i \in [1..N]\}$ the training set.
- A classifier induced from the training set $\mathcal{L}$ will be denoted $\psi(\cdot, \mathcal{L})$. It is a function that for any vector x_i from $\mathcal{X}$ associates a class:

$$\psi(\cdot, \mathcal{L}) : \mathcal{X} \rightarrow [1..K]$$

- Applying ψ on a new object x will therefore return a class prediction:

$$\hat{y} = \psi(x, \mathcal{L})$$

Types of models and classifiers

How are patterns and models expressed ? There are 2 extremes:

- **Black box representation**: The model, structure or function is impossible to grasp for a human unfamiliar with the generating algorithm.
  - Examples: Deep learning algorithms
- **White box representation**: The model and its construction process are easy to understand and reveal which kind of structure to expect.
  - Examples: Decision trees, K-Nearest neighbors

Another distinction:

- **Generative** models learn how each class generates data, $p(x \mid c)$ (e.g. Naive Bayes).
- **Discriminative** models learn the decision directly, $p(c \mid x)$ or $\psi(x)$ (e.g. decision trees).

The 5 steps of supervised learning

1. Decide on a training set that will be representative of the real-world use of your classifier.
2. Determine your input features and the representations that you want to use in your model.
3. Decide on the structure of your learning function, and choose a supervised algorithm compatible with this model.
4. Run the algorithm on your training set. Use a validation set or cross-validation (which works with any algorithm) to tune your model.
5. Evaluate the performance of your final model once, on a separate test set.

The bias-variance trade-off

Supervised learning faces several challenges.

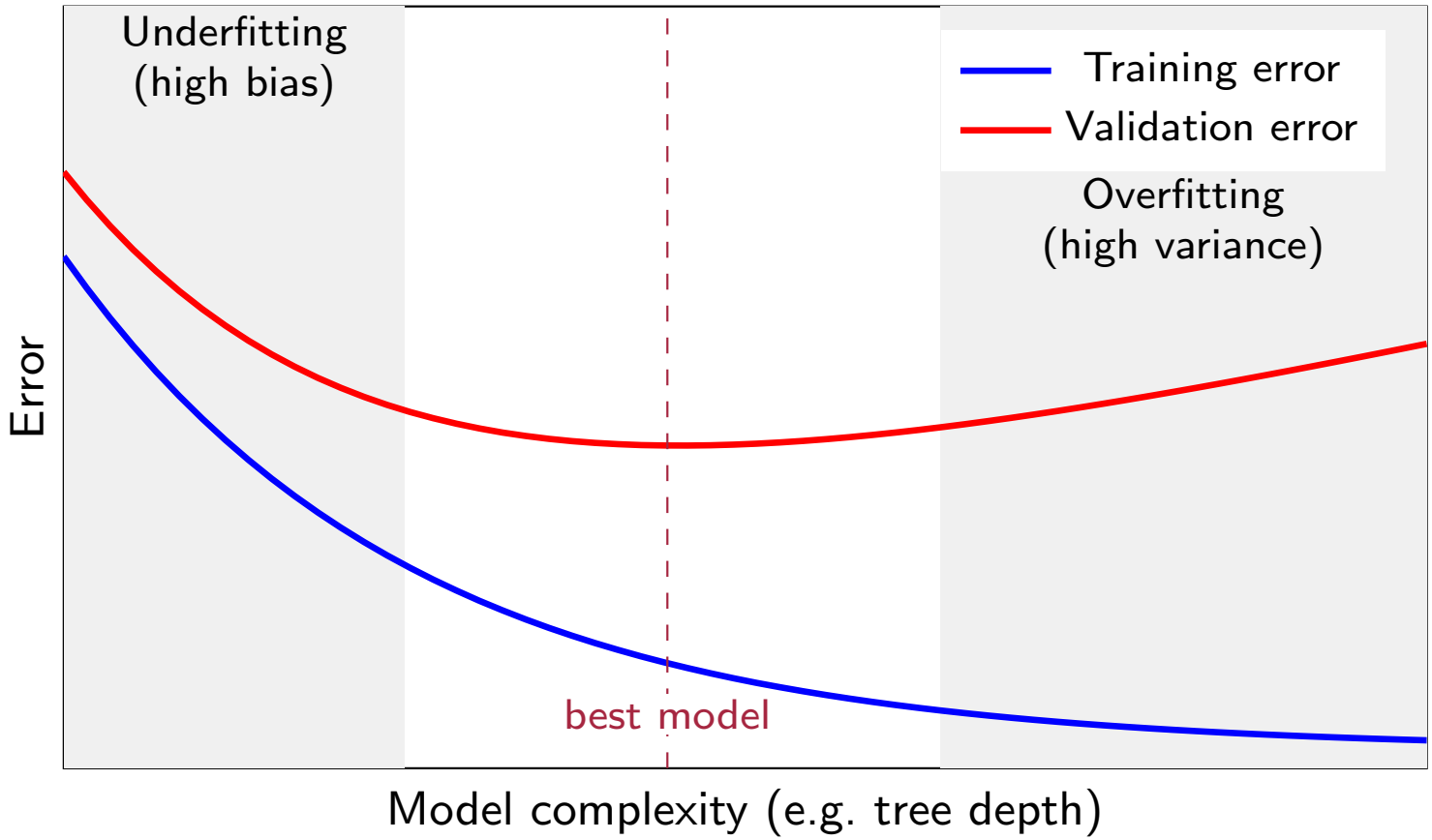
The bias-variance trade-off in supervised learning

The bias-variance trade-off is a problem in which a supervised algorithm has to achieve simultaneously two seemingly incompatible goals:

- Fitting the training data well (low bias).
- Generalizing to data outside of the training set (low variance).

- The **bias** is the error from erroneous assumptions in the learning algorithm or model that usually results in the algorithms missing important links between the input variables and the output, thus leading to **underfitting**.
- The **variance** is the error caused by a too high sensitivity of the model toward small variations in the training set. This results in **overfitting** and the model being unable to generalize to data outside of the training set.

Underfitting and overfitting



Expected error = Bias^2 + Variance + irreducible noise. Choose the complexity that minimises the **validation** error.

Function complexity and amount of training data

The second issue is the available amount of training data when compared to the relative complexity of the real model to be learned:

- If the model is simple, a learning algorithm with a high bias and a low variance should be able to learn it from a small amount of data.
- If the model is complex, it will only be learnable from a very large amount of training data and using a learning algorithm with a low bias and a high variance.

Remark

Good learning algorithms should be able to adjust their bias-variance trade-off based on the amount of available data and the apparent complexity of the model to be learned.

Picking the right input variables

The problem with too many input variables

Even if the real learning model depends only on a very small number of variables, the algorithm may never figure it out if it is flooded with a very high number of input variables.

- The result may end up being a very complex and overfitting model.
- Models with too many variables cannot easily be understood and interpreted.

- A good understanding of your data and of the problem that you want to model will help remove irrelevant features.
- Scaling your data may have a huge influence on the results of distance-based methods (e.g. k-NN). Decision trees are not affected by scaling.
- It is important to check for correlation between the attributes and to remove redundant variables.

Outline

1 Introduction

2 Decision trees

3 Naive Bayes Classifiers

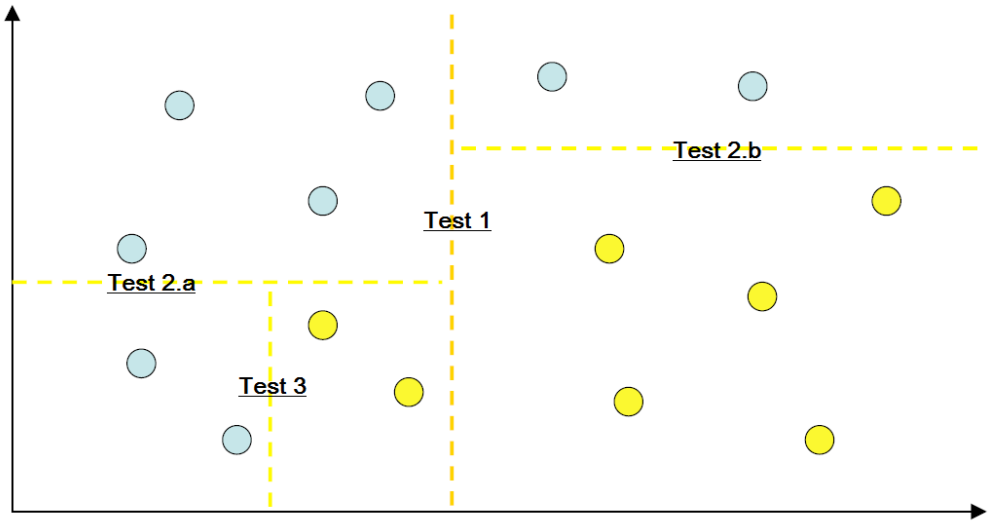
4 Comparison and summary

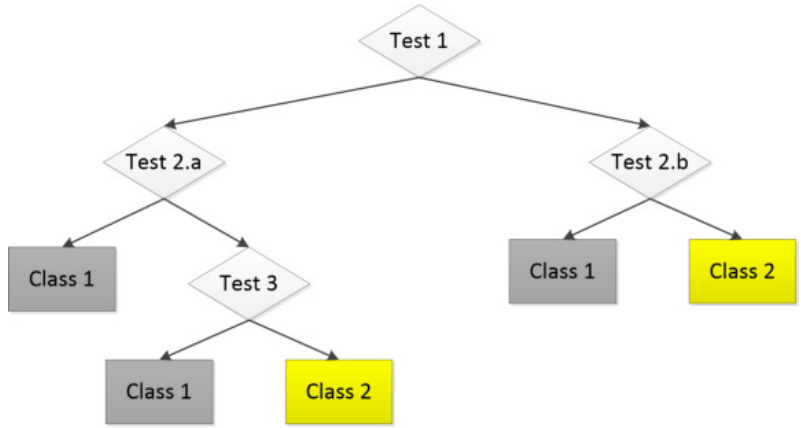
5 Bibliography

Decision trees

Decision trees are very common classifiers that mine rules from the training set:

- They are mostly applied to categorical data, but not only.
- They decompose the feature space according to the most discriminating variable at each stage.
- There are usually more than one possible tree per data set.





Decision trees

A decision tree is a tree-structured representation of a discrete-valued function. It can be used as a decision support tool that uses a tree-like graph or model of decisions and their possible consequences. Learning decision trees are among the most commonly used classification methods. The main algorithms are ID3, C4.5/C5.0 (Quinlan) and CART (Breiman et al.).

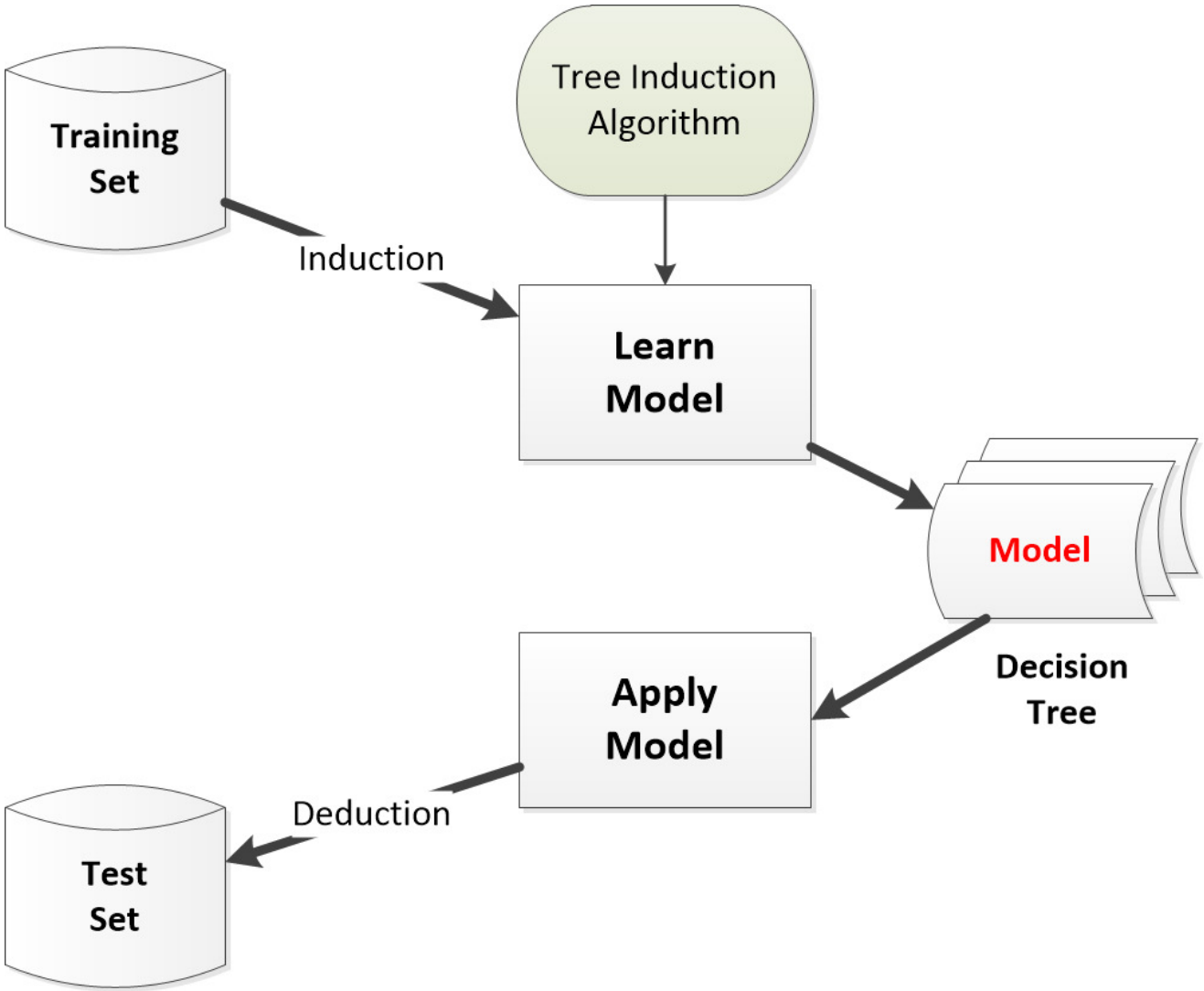
Properties

- **Expressiveness:** It can represent disjunctions of conjunctions
- **Readability:** It can be translated as a set of decision rules

Notation

- Disjunction : A or B
- Conjunction : A and B

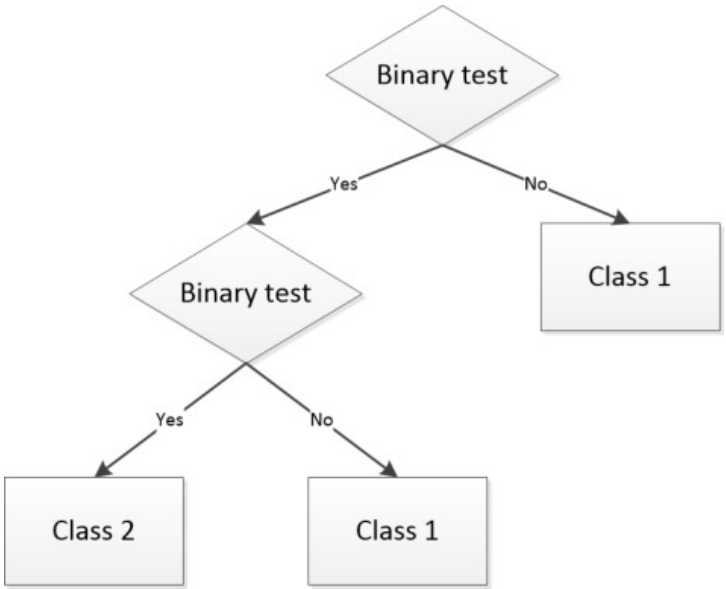
Decision tree: Classification Process



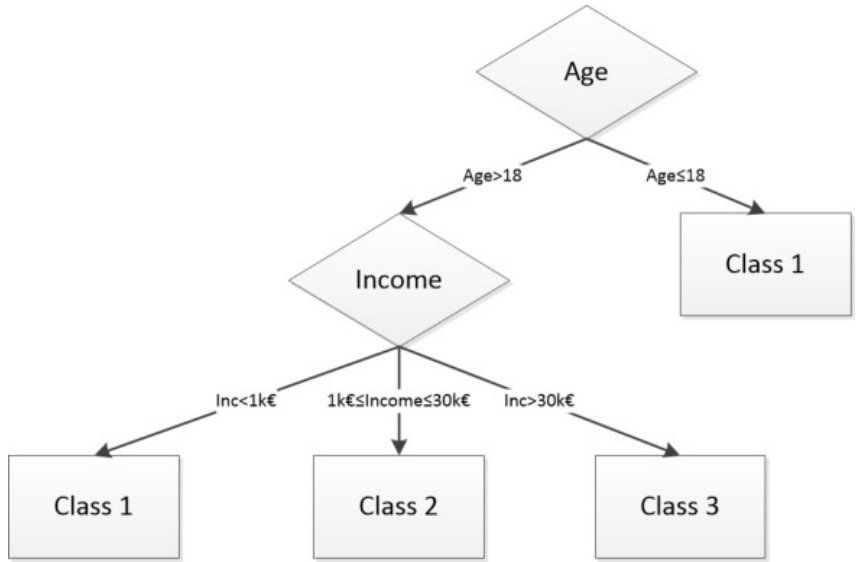
Types of trees

Decision trees can be categorized according to three criteria:

- The type of data: Numerical, Categorical, Mixed
- The type of splits: binary splits, multi-way splits
- The overall shape of the tree

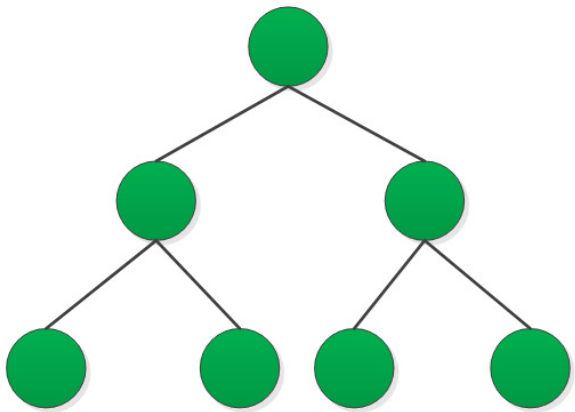


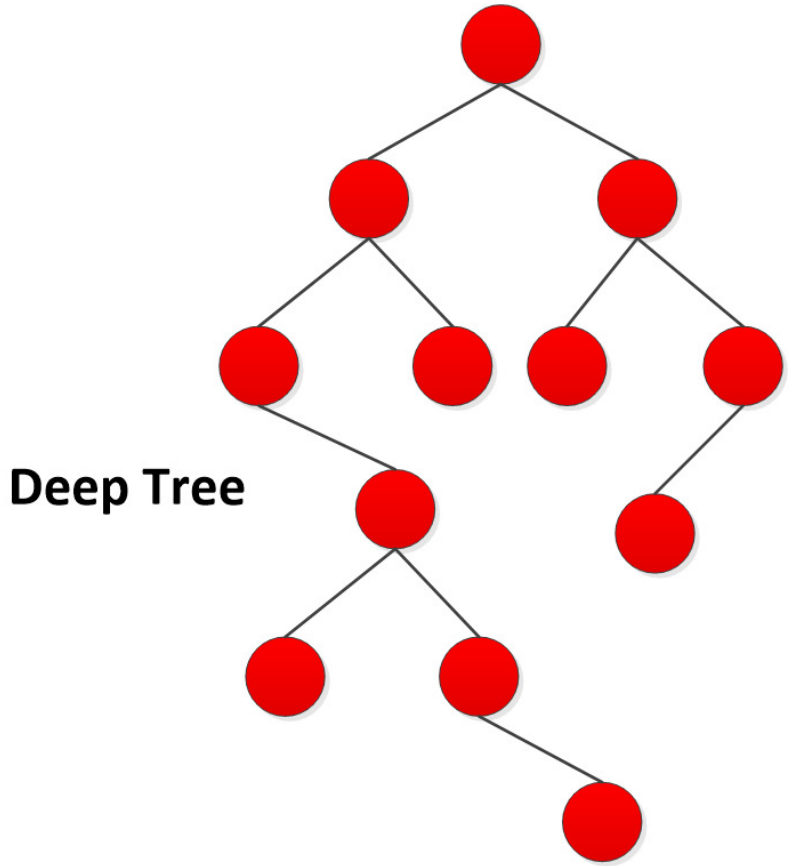
(a) Example of a binary tree

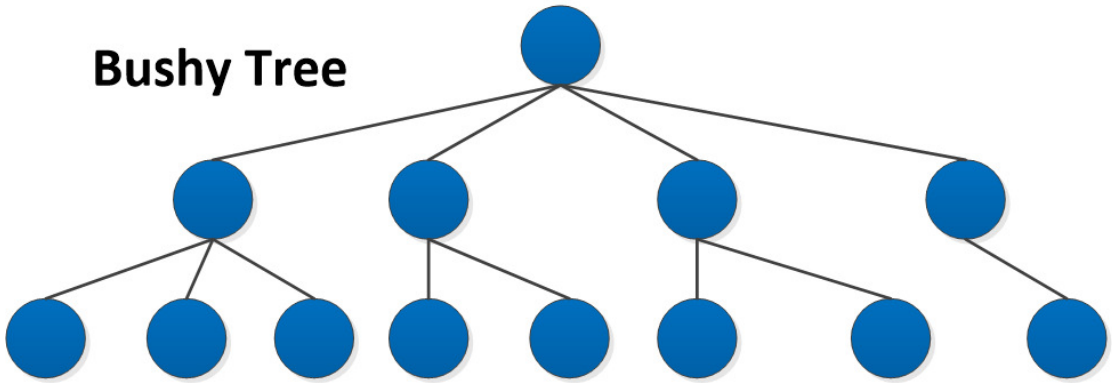


(b) Example of a numerical multi-way tree

Types of trees







Building a tree: greedy top-down induction

All classical algorithms (ID3, C4.5, CART) build the tree recursively:

Algorithm BuildTree(S)

1. If all examples of S have the same class, or a stopping criterion is met (maximum depth, too few examples), return a **leaf** labeled with the majority class.
2. Otherwise, choose the attribute A that best separates the classes of S.
3. Create a node testing A, with one branch per value (or per interval) of A.
4. For each branch v, call BuildTree(S_v) on the examples with $A = v$.

- The choice is **greedy**: the best split is chosen locally, the tree is never revised.
- "Best separates" is measured with an **impurity** measure: entropy or Gini index.

Entropy and information gain (ID3, C4.5)

Entropy of a set S with K classes

$$H(S) = -\sum_{k=1}^{K} p_k \log_2 p_k$$

where p_k is the proportion of examples of class k in S.

- $H(S) = 0$: the node is pure. $H(S) = 1$: two classes, 50% / 50%.

Information gain of attribute A

$$IG(S, A) = H(S) - \sum_{v \in \text{values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

- The attribute with the **highest information gain** is chosen for the split.

Gini index (CART) and gain ratio (C4.5)

Gini impurity

$$Gini(S) = 1 - \sum_{k=1}^{K} p_k^2$$

CART chooses the binary split that maximises the decrease in Gini impurity.

Gain ratio

Information gain favours attributes with many values: an ID column would give pure leaves and a maximal gain. C4.5 normalises the gain:

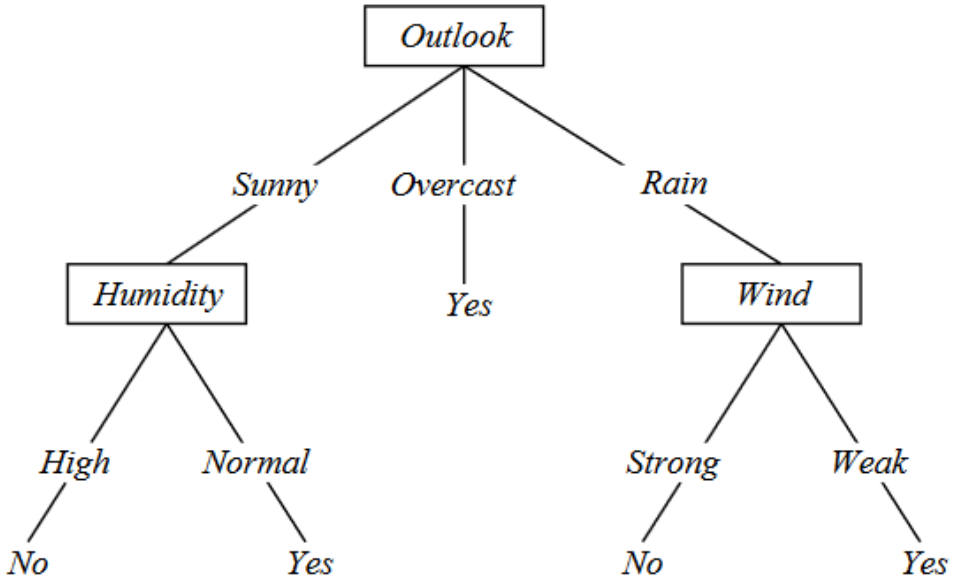
$$GR(S, A) = \frac{IG(S, A)}{-\sum_v \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}}$$

In practice entropy and Gini almost always produce very similar trees.

The Weather Problem

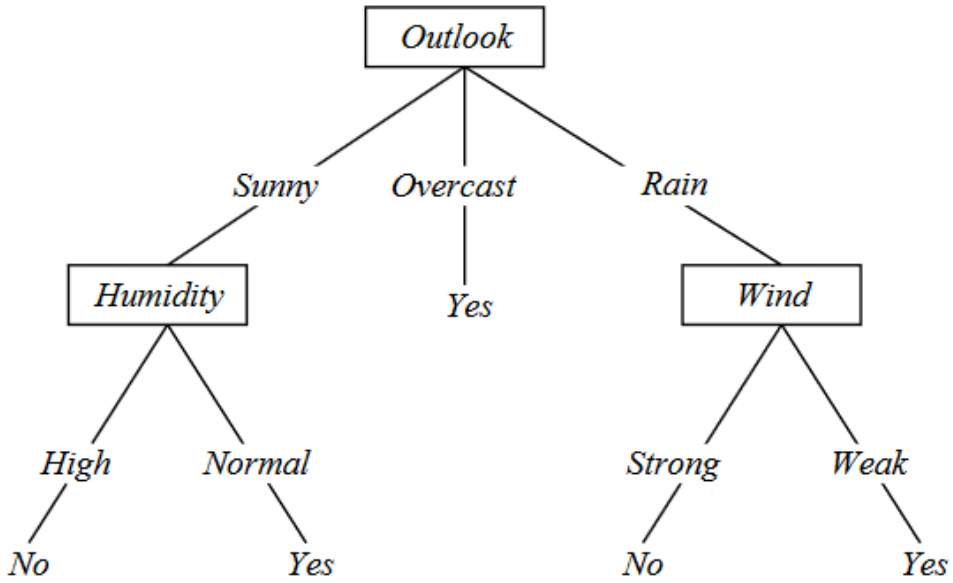
Outlook	Temperature	Humidity	Wind	Play
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
...	...	...	...	...

The goal of this problem is to determine which weather conditions are the most favorable to let your kids go play outside.



From the resulting tree, we can see that temperature is not among the most relevant parameters here.

The Weather Problem



From this tree we can extract the following rules:

- If (Outlook=Sunny) AND (Humidity=High) THEN Play=No
- If (Outlook=Sunny) AND (Humidity=Normal) THEN Play=Yes
- If (Outlook=Overcast) THEN Play=Yes
- If (Outlook=Rain) AND (Wind=Strong) THEN Play=No
- If (Outlook=Rain) AND (Wind=Weak) THEN Play=Yes

The Weather Problem: the full data

Outlook	Temp.	Humidity	Wind	Play
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

14 examples: 9 Yes, 5 No.

$$H(S) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14}$$
$$= 0.940$$

Why is **Outlook** at the root of the tree? Let us compute the information gain of each attribute.

The Weather Problem: choosing the root

Example for Outlook: Sunny (2 Yes, 3 No), Overcast (4 Yes, 0 No), Rain (3 Yes, 2 No).

$$IG(S, \text{Outlook}) = 0.940 - \tfrac{5}{14}\,0.971 - \tfrac{4}{14}\,0 - \tfrac{5}{14}\,0.971 = 0.247$$

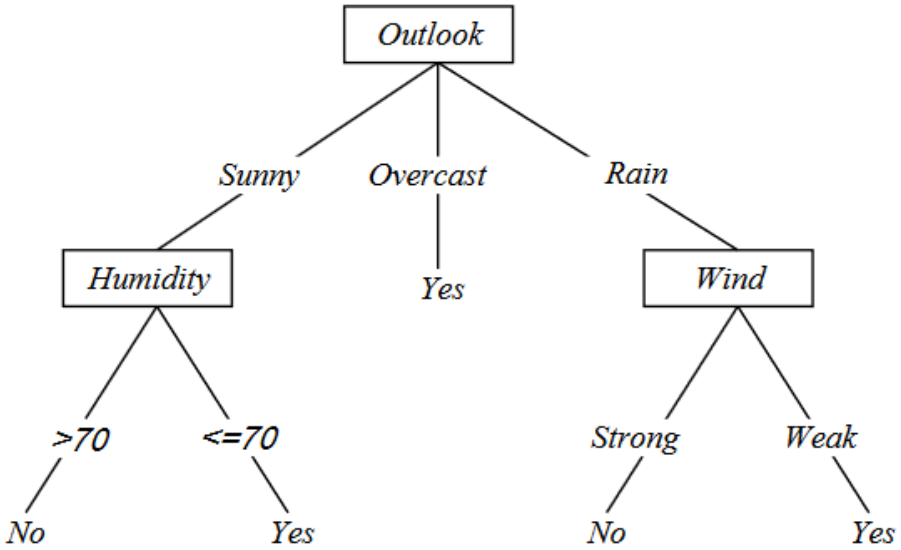
Attribute	Information gain	Gini decrease
Outlook	**0.247**	**0.116**
Humidity	0.152	0.092
Wind	0.048	0.031
Temperature	0.029	0.019

- Outlook is chosen as the root by both criteria; the Overcast branch is already pure.
- In the Sunny branch: $IG(\text{Humidity}) = 0.971$ vs 0.571 (Temp.) and 0.020 (Wind).
- Temperature has the lowest gain everywhere: it never enters the tree.

The Weather Problem

Outlook	Temperature	Humidity	Wind	Play
Sunny	29	85	Weak	No
Sunny	27	90	Strong	No
Overcast	28	86	Weak	Yes
Rain	24	80	Weak	Yes
...	...	...	...	...

The same problem can be processed with mixed attributes. A similar tree can be found.

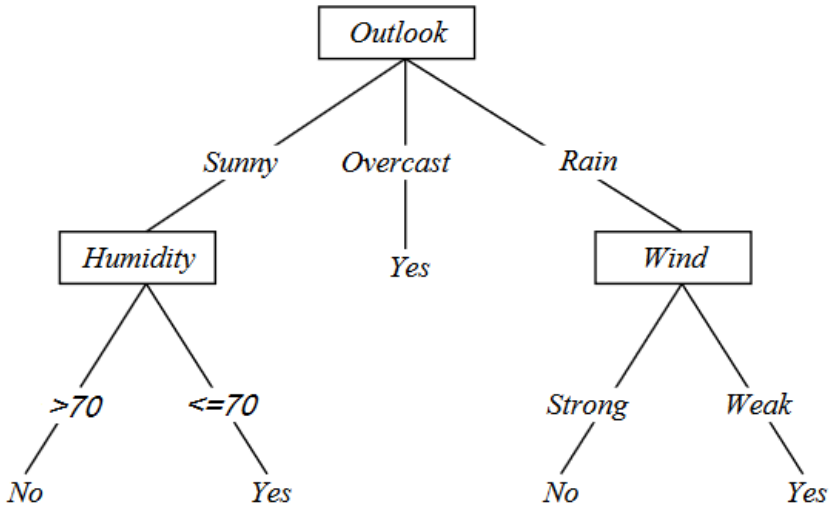


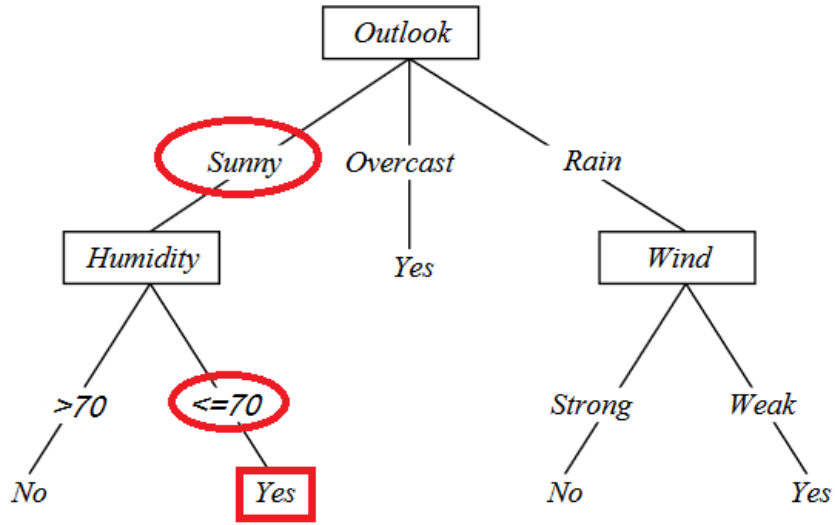
Remark

It usually takes more time to compute a decision tree with numerical values, because every possible cut value must be evaluated to find the optimal one.

The Weather Problem: Unlabeled Example

Outlook	Temperature	Humidity	Wind	Play
Sunny	25	55	Weak	?



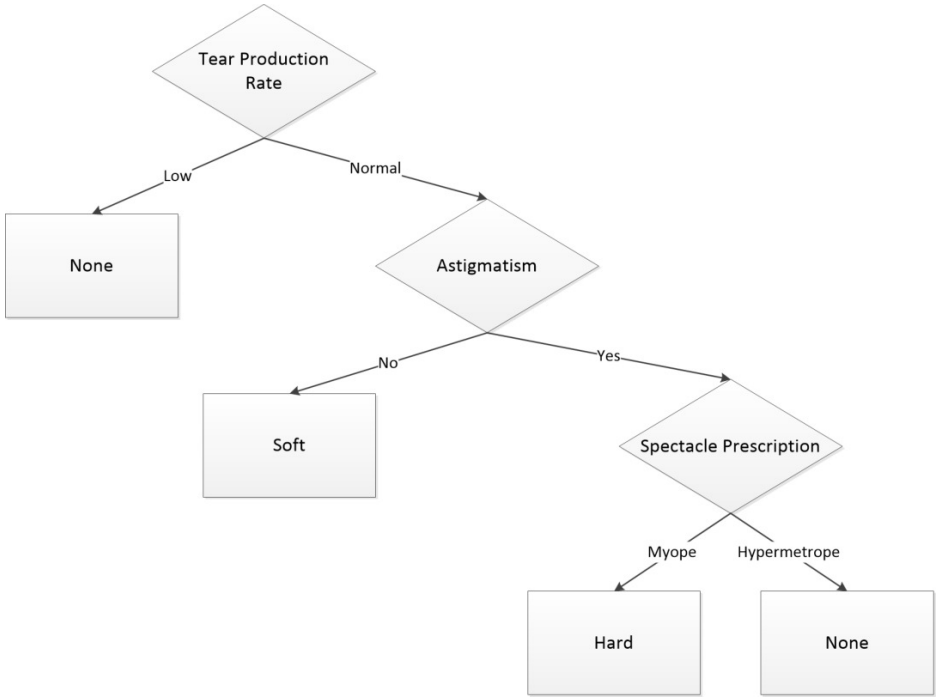


This example would be labeled **Yes** by the decision tree.

The Contact Lenses Data

Age	Prescription	Astigmatism	Tear production	Recommended Lenses
Young	Myope	No	Low	None
Young	Myope	No	Normal	Soft
Young	Myope	Yes	Low	None
Young	Myope	Yes	Normal	Hard
Young	Hypermetrope	No	Low	None
Young	Hypermetrope	No	Normal	Soft
Young	Hypermetrope	Yes	Low	None
Young	Hypermetrope	Yes	Normal	Hard
Pre-presbyopic	Myope	No	Low	None
Pre-presbyopic	Myope	No	Normal	Soft
Pre-presbyopic	Myope	Yes	Low	None
Pre-presbyopic	Myope	Yes	Normal	Hard
Pre-presbyopic	Hypermetrope	No	Low	None
Pre-presbyopic	Hypermetrope	No	Normal	Soft
Pre-presbyopic	Hypermetrope	Yes	Low	None
Pre-presbyopic	Hypermetrope	Yes	Normal	None
Presbyopic	Myope	No	Low	None
Presbyopic	Myope	No	Normal	None
Presbyopic	Myope	Yes	Low	None
Presbyopic	Myope	Yes	Normal	Hard
Presbyopic	Hypermetrope	No	Low	None
Presbyopic	Hypermetrope	No	Normal	Soft
Presbyopic	Hypermetrope	Yes	Low	None
Presbyopic	Hypermetrope	Yes	Normal	None

The Contact Lenses Data



This small tree misclassifies 2 of the 24 training examples (Young/Hypermetrope/Astigmatic/Normal → Hard, and Presbyopic/Myope/Not astigmatic/Normal → None). It trades 2 errors for a much simpler tree: see pruning.

Setting the depth of a tree

- Deep trees have **high variance**: they fit the noise of their training set, can't generalize much outside of it and are difficult to interpret.

Setting the right Depth for your tree

Most Decision trees algorithms will allow you to choose the maximum depth of your tree.

- How deep is too deep will depend on the complexity of the problem
- Deeper trees tend towards overfitting, while less deep trees will tend towards underfitting.
- The best option is to start from a deep tree and to prune it in a way that minimizes the error on a held-out **validation set**.

- While balanced-trees are usually the most preferable option, bushy trees should not be frowned upon in problems with a lot of classes, or when they can help reducing the depth of the tree.

Pruning

- **Pre-pruning** (early stopping): stop growing a branch when it reaches a maximum depth, contains too few examples, or when the best split improves impurity too little.
- **Post-pruning**: grow the full tree, then replace subtrees by leaves.
  - *Reduced-error pruning*: remove a subtree if the validation error does not increase.
  - *Cost-complexity pruning* (CART): minimise $\text{Error}(T) + \alpha\,|\text{leaves}(T)|$, with α chosen by cross-validation.

Remark

Pruning never decreases the training error. Its purpose is to decrease the error on unseen data.

Summary

Pros

- Intuitive, easy to understand and to use
- Build comprehensible models that translate into rules
- One of the most commonly used classifiers for decision making
- No feature scaling needed; handles numerical and categorical data

Cons

- There are always several possible trees, and greedy induction does not guarantee the best one
- Unstable: a small change in the data can give a very different tree (high variance)
- Choosing the depth of the tree is a complex decision
- Splits are axis-parallel: oblique boundaries need many nodes

Ensembles of trees (random forests, boosting) reduce the variance of single trees.

Outline

1 Introduction

2 Decision trees

3 Naive Bayes Classifiers

4 Comparison and summary

5 Bibliography

Naive Bayes Classifiers: Introduction

Naive Bayes classifiers are a family of simple probabilistic classifiers based on Bayes Theorem and a strong independence hypothesis between the features.

Intuition

To find out the probability of the previously unseen instance belonging to each class, simply pick the "most probable" class.

- These probabilities are estimated using Bayes theorem applied on the training data.

Naive Bayes Classifiers: Introduction

Bayes Theorem

$$p(c_j \mid x) = \frac{p(x \mid c_j)\, p(c_j)}{p(x)}$$

- $p(c_j \mid x)$ The probability of instance x belonging to class c_j.
  - It is the *posterior probability*. We want to compute it.
- $p(x \mid c_j)$ The probability of generating instance x knowing class c_j.
  - Knowing the distribution function of class c_j and the features of x, what is the *likelihood* of x ?
- $p(c_j)$ The *a priori* probability of class c_j.
  - How frequent is class c_j in the training set ?
- $p(x)$ The occurrence probability of instance x, also called *evidence*.
  - It is the same for every class, so it can be ignored when comparing classes. It is only needed as a normalisation constant: $p(x) = \sum_j p(x \mid c_j)\, p(c_j)$.

Naive Bayes Classifiers: Simple example

One year ago, on my way back from Holland, I was arrested by a police officer named "Claude". I was a bit high and can't remember whether Officer Claude was a male or a female ...

Using a bayesian classifier, and a police data base with names and sex, we can try to guess whether it is more likely that officer Claude was a male or a female. We have two classes: $c_1 = male$ and $c_2 = female$.

What is the probability of being named "Claude" given that you're a male

What's the probability of being a male ?

$$p(male|Claude) = \frac{p(Claude|male)p(male)}{p(Claude)}$$

What's the probability of being named "Claude" ?

Note: In French, "Claude" can be a male or a female name



Claude Gensac



Claude François

Naive Bayes Classifiers: Simple example

Name	Sex
Claude	Male
Laura	Female
Claude	Female
Claude	Female
Arthur	Male
Karima	Female
Rose	Female
Sergio	Male

Table 3: Training data (List of Police officers in Lille)

$$p(\textit{male} \mid \textit{Claude}) = \frac{p(\textit{Claude} \mid \textit{male})\, p(\textit{male})}{p(\textit{Claude})}$$
$$= \frac{1/3 \times 3/8}{3/8} = \frac{0.125}{0.375} = \frac{1}{3}$$

$$p(\textit{female} \mid \textit{Claude}) = \frac{p(\textit{Claude} \mid \textit{female})\, p(\textit{female})}{p(\textit{Claude})}$$
$$= \frac{2/5 \times 5/8}{3/8} = \frac{0.250}{0.375} = \frac{2}{3}$$

The two posteriors sum to 1. Since $1/3 < 2/3$, we can conclude that most likely Officer Claude was a female !

Naive Bayes with several features

- In the previous example there was only one feature: the name. What happens when there are more ?
- To make the problem simpler, naive Bayes classifiers assume that the attributes are **conditionally independent given the class** (which is not always true).

Independence Hypothesis

- Let us note $\mathbf{x} = \{x_1, \ldots, x_d\}$ a data with d features.
- Under the hypothesis that all attributes are independent given the class, we can write:

$$p(\mathbf{x} \mid c_j) = p(x_1 \mid c_j) \times p(x_2 \mid c_j) \times \cdots \times p(x_d \mid c_j)$$

Therefore, we have:

$$p(c_j \mid \mathbf{x}) = \frac{p(c_j) \prod_{i=1}^{d} p(x_i \mid c_j)}{p(\mathbf{x})} \propto p(c_j) \prod_{i=1}^{d} p(x_i \mid c_j)$$

Naive Bayes with several features

- Note that Naive Bayes **is not sensitive** to irrelevant features.

Suppose that we are trying to classify persons gender based on several features, including eye color (which is irrelevant). For Jessica's feature vector $\mathbf{x}_J$:

$$p(\mathbf{x}_J \mid c_j) = p(eye = brown \mid c_j) \times p(wears_dress = yes \mid c_j) \times \cdots$$

$$p(\mathbf{x}_J \mid female) = 9000/10000 \times 7500/10000 \times \cdots$$
$$p(\mathbf{x}_J \mid male) = 9001/10000 \times 3/10000 \times \cdots$$

$p(eye = brown \mid female)$ and $p(eye = brown \mid male)$ should be almost identical and won't affect the outcome much. Wearing a dress however ...

Naive Bayes with several features

- Note that Naive Bayes **is not sensitive** to irrelevant features.

Suppose that we are trying to classify persons gender based on several features, including eye color (which is irrelevant). For Jessica's feature vector $\mathbf{x}_J$:

$$p(\mathbf{x}_J \mid c_j) = p(eye = brown \mid c_j) \times p(wears_dress = yes \mid c_j) \times \cdots$$

$$p(\mathbf{x}_J \mid female) = 9000/10000 \times 7500/10000 \times \cdots$$
$$p(\mathbf{x}_J \mid male) = 9001/10000 \times 3/10000 \times \cdots$$

$p(eye = brown \mid female)$ and $p(eye = brown \mid male)$ should be almost identical and won't affect the outcome much. Wearing a dress however ...

Remark

This assumes that the estimates of the probabilities are good enough. Therefore the training set must be as big and as unbiased as possible.

The zero-frequency problem and Laplace smoothing

If a value never occurs with a class in the training set, its estimate is 0 and the whole product becomes 0, whatever the other features say.

- Weather data: Outlook = Overcast never occurs with Play = No, so $p(\text{Overcast} \mid No) = 0/5$ and $p(No \mid \text{Overcast}, \ldots) = 0$.

Laplace (additive) smoothing

$$p(x_i = v \mid c_j) = \frac{n_{v,j} + \alpha}{n_j + \alpha\,|V_i|}$$

$n_{v,j}$: count of value v in class c_j; n_j: size of class c_j; $|V_i|$: number of values of attribute i; usually $\alpha = 1$.

- Overcast with $\alpha = 1$: $p(\text{Overcast} \mid No) = (0 + 1)/(5 + 3) = 0.125$.
- With many features, compute $\log p(c_j) + \sum_i \log p(x_i \mid c_j)$ to avoid numerical underflow.

Naive Bayes variants

The form of $p(x_i \mid c_j)$ depends on the type of the features:

Variant	Features	$p(x_i \mid c_j)$	scikit-learn
Categorical	discrete values	frequency tables	`CategoricalNB`
Gaussian	real numbers	normal law $\mathcal{N}(\mu_{ij}, \sigma_{ij})$	`GaussianNB`
Multinomial	counts (words)	multinomial	`MultinomialNB`
Bernoulli	binary	Bernoulli	`BernoulliNB`

- The Officer Claude and weather examples use categorical Naive Bayes.
- The mosquito example that follows uses **Gaussian** Naive Bayes.
- Multinomial Naive Bayes is a classic baseline for spam filtering and text classification.

Naive Bayes on the Weather Problem

New day: Outlook = Sunny, Temperature = Cool, Humidity = High, Wind = Strong. Play?

	Play = Yes	Play = No
$p(c)$	9/14	5/14
$p(\text{Sunny} \mid c)$	2/9	3/5
$p(\text{Cool} \mid c)$	3/9	1/5
$p(\text{High} \mid c)$	3/9	4/5
$p(\text{Strong} \mid c)$	3/9	3/5
Product $\propto p(c \mid \mathbf{x})$	0.0053	0.0206
Normalised	20.5%	**79.5%**

Naive Bayes predicts **No**. The decision tree gives the same answer (Sunny and High humidity $\rightarrow$ No), but Naive Bayes also gives a probability.

Naive Bayes: Properties

Pros

- The only things to store are the probabilities: The training data need not be kept in memory and a single scan of the data is necessary to acquire the probabilities.
- The model is quite simple to understand.
- One of the fastest prediction models.

Naive Bayes: Properties

Pros

- The only things to store are the probabilities: The training data need not be kept in memory and a single scan of the data is necessary to acquire the probabilities.
- The model is quite simple to understand.
- One of the fastest prediction models.

Cons

- Naive Bayes assumes that the features are conditionally independent. It is usually not true and can lead to more or less bias when several of them are too correlated.
- Probability estimates depend on the training data: a value with a frequency of 0 in the training set sets the whole product to 0. Laplace smoothing is required.
- The posterior probabilities are often poorly calibrated, even when the predicted class is right.

Mosquito identification

In this example we consider 3 species of mosquitoes:

- *Culex pipiens*, the common house mosquito
- *Anopheles stephensi*, a malaria vector from South Asia and the Middle East
- *Aedes aegypti*, the yellow fever mosquito (may also carry Dengue fever, Zika, or Chikungunya)



(a) *Culex pipiens*



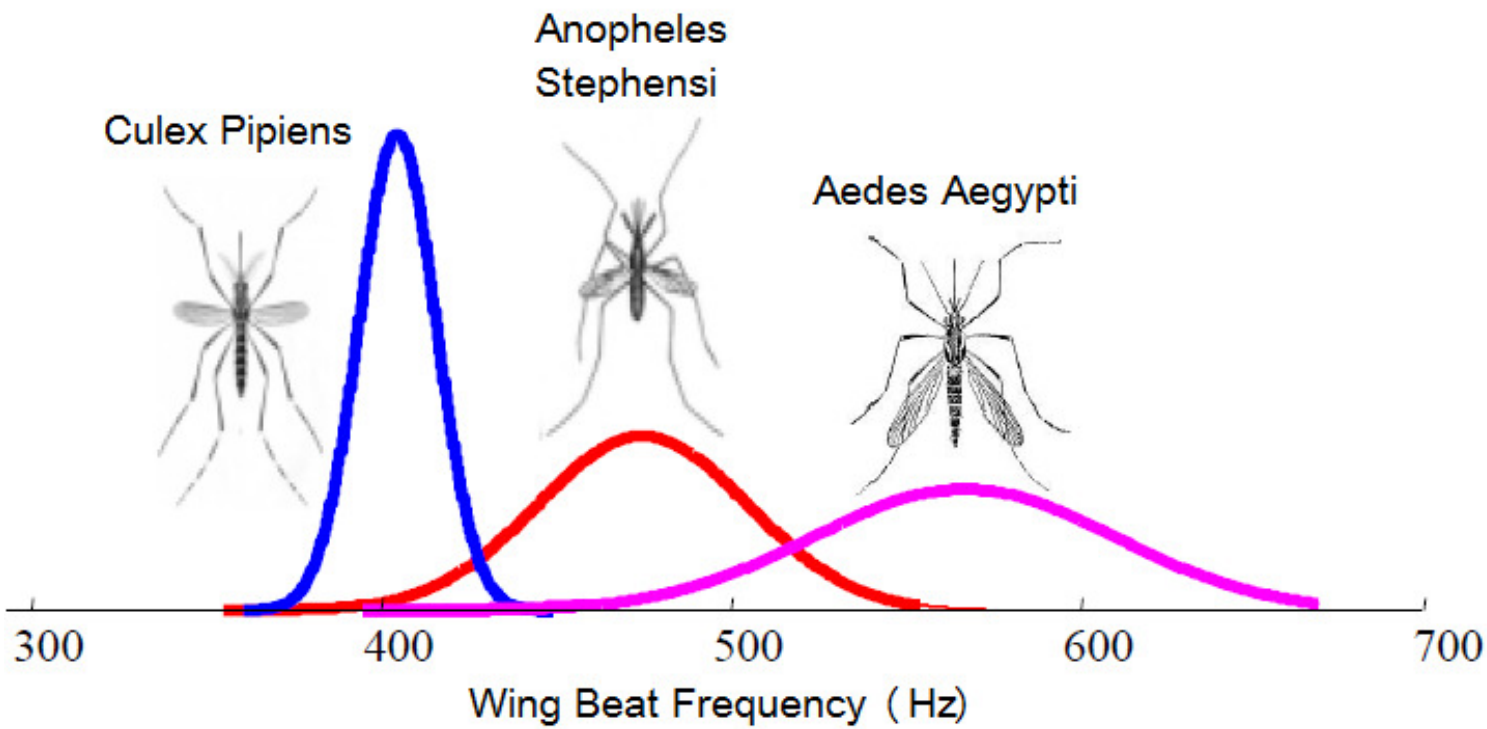
(b) *Anopheles stephensi*



(c) *Aedes aegypti*

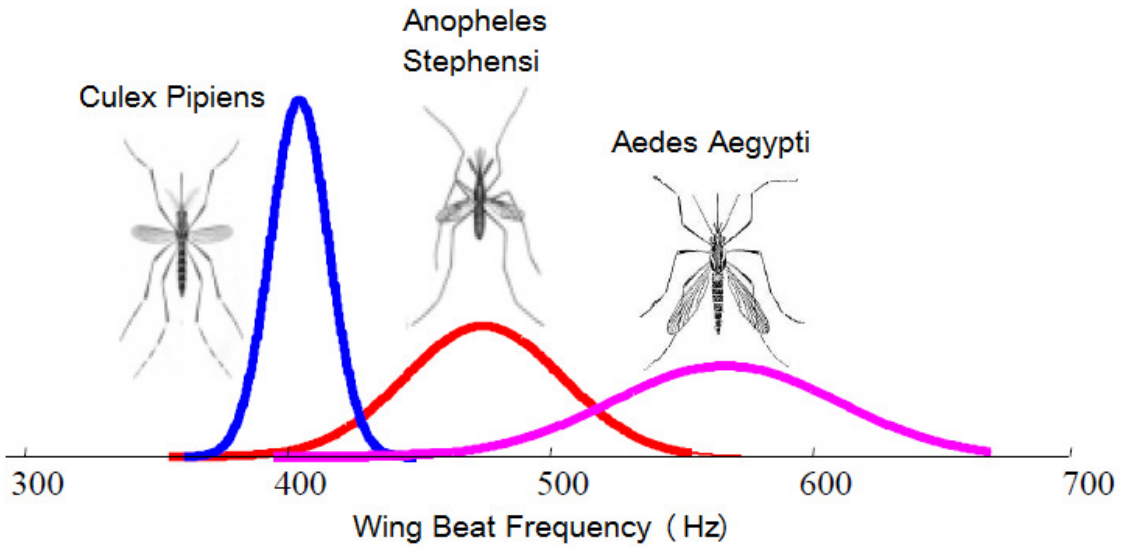
Mosquito identification

Mosquitoes have distinct wing beat frequencies.



- *Culex pipiens*: $\mathcal{N}(\mu = 390, \sigma = 14)$
- *Anopheles stephensi*: $\mathcal{N}(\mu = 475, \sigma = 30)$
- *Aedes aegypti*: $\mathcal{N}(\mu = 567, \sigma = 43)$

Mosquito identification



- *Culex*: $\mathcal{N}(\mu = 390, \sigma = 14)$
- *Anopheles*: $\mathcal{N}(\mu = 475, \sigma = 30)$
- *Aedes*: $\mathcal{N}(\mu = 567, \sigma = 43)$

Suppose I see a mosquito with a wing frequency of 500Hz, which one is it?

Remark

It is $p(\textit{wingbeat} \mid \textit{species})$ that follows the normal law and not the other way around !

Mosquito identification

Suppose I see a mosquito with a wing frequency of 500Hz, which one is it?

Mosquito identification

Suppose I see a mosquito with a wing frequency of 500Hz, which one is it?

Assuming the three species are equally common (equal priors):

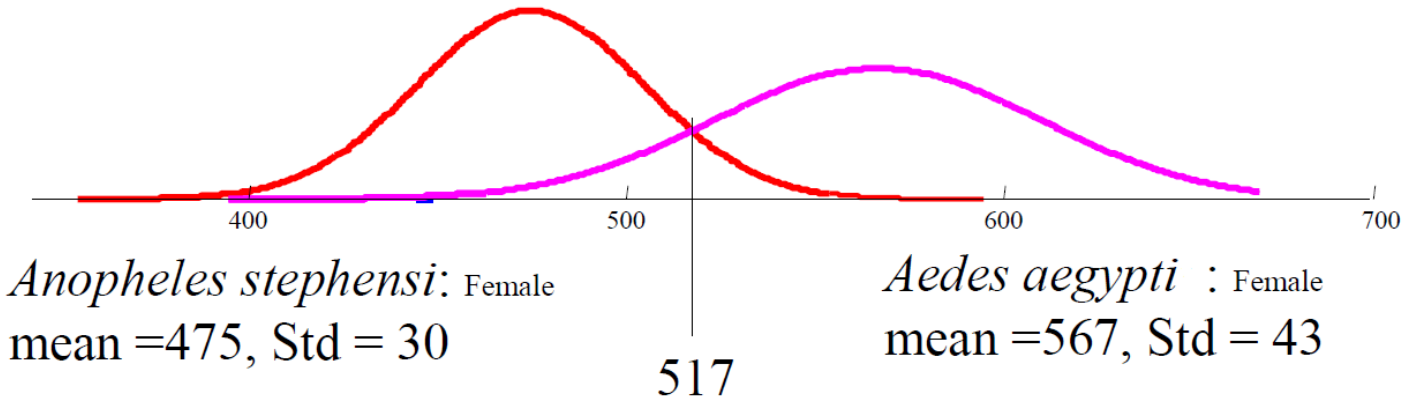
$$p(\textit{Culex} \mid \textit{wingbeat} = 500) \propto \frac{1}{14\sqrt{2\pi}} e^{-\frac{(500-390)^2}{2\times 14^2}} \approx 0$$

$$p(\textit{Anopheles} \mid \textit{wingbeat} = 500) \propto \frac{1}{30\sqrt{2\pi}} e^{-\frac{(500-475)^2}{2\times 30^2}} = 0.0094$$

$$p(\textit{Aedes} \mid \textit{wingbeat} = 500) \propto \frac{1}{43\sqrt{2\pi}} e^{-\frac{(500-567)^2}{2\times 43^2}} = 0.0028$$

Without further information, it is most likely an *Anopheles*.

Mosquito identification: Getting the probability



These do not look like probabilities:

$p(Anopheles \mid wbf = 500) \propto 0.0094$

$p(Aedes \mid wbf = 500) \propto 0.0028$

They are densities: we must normalise them.

Getting the probabilities

$$P(Anopheles \mid wbf = 500) = \frac{0.0094}{0.0122} = 0.77$$

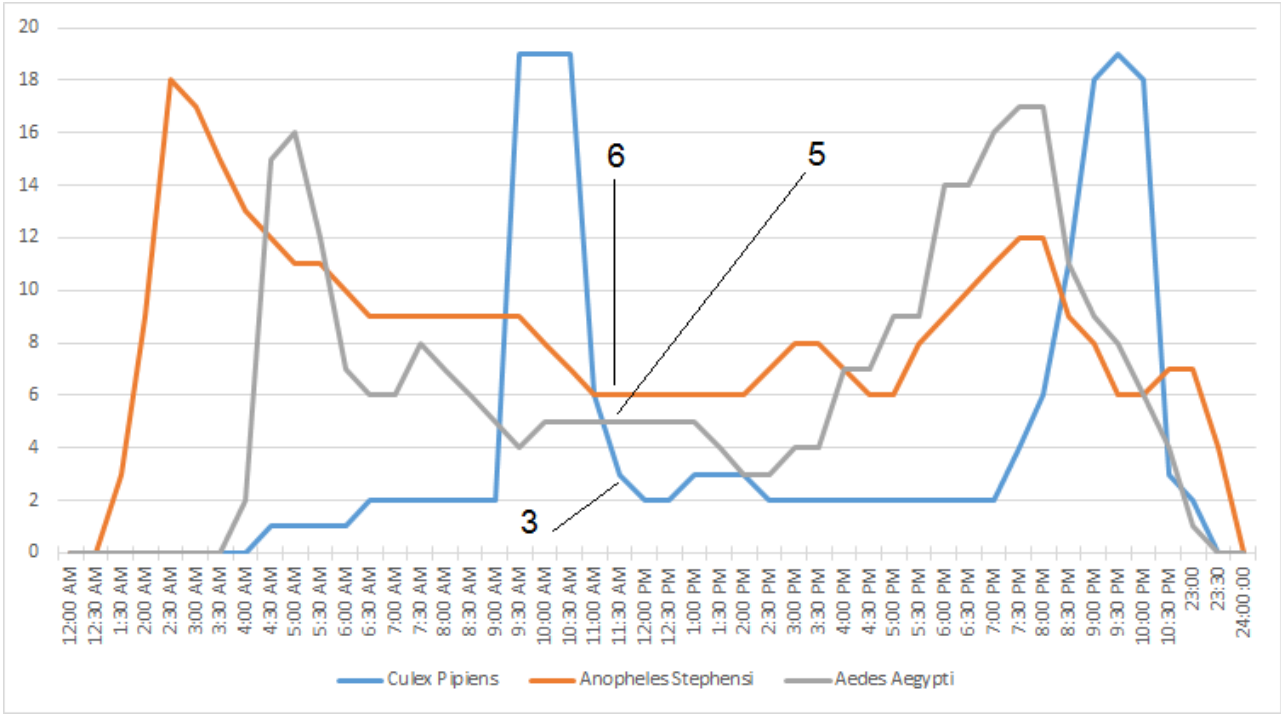
$$P(Aedes \mid wbf = 500) = \frac{0.0028}{0.0122} = 0.23$$

with $0.0122 = 0 + 0.0094 + 0.0028$

The two curves cross at 517 Hz: below it, *Anopheles* is more likely.

Mosquito identification: More features

We now have additional information in the form of a chart representing how many mosquitoes of each species are active depending on the time of the day.



Suppose I am savagely attacked by a mosquito with a wingbeat frequency of 420Hz at 11:30am. Which one is the most likely culprit? The chart counts all active mosquitoes, so it already tells us how common each species is at 11:30.

Mosquito identification: More features

Assuming wingbeat and time are independent given the species:

$$P(c \mid 420\text{Hz}, 11{:}30) \propto P(c)\, p(420 \mid c)\, P(11{:}30 \mid c) \propto p(420 \mid c)\, P(c \mid 11{:}30)$$

Remark: Bayes Theorem replaces $P(c)\, P(11{:}30 \mid c)$ by $P(c \mid 11{:}30)\, P(11{:}30)$. From the chart: $3 + 6 + 5 = 14$ active mosquitoes at 11:30.

Species	$p(420 \mid c)$	$P(c \mid 11{:}30)$	Product
Culex	0.00287	$3/14$	6.1×10^{-4}
Anopheles	0.00248	$6/14$	1.06×10^{-3}
Aedes	0.000027	$5/14$	9.6×10^{-6}

Mosquito identification: More features

Assuming wingbeat and time are independent given the species:

$$P(c \mid 420\text{Hz}, 11{:}30) \propto P(c)\, p(420 \mid c)\, P(11{:}30 \mid c) \propto p(420 \mid c)\, P(c \mid 11{:}30)$$

Remark: Bayes Theorem replaces $P(c)\, P(11{:}30 \mid c)$ by $P(c \mid 11{:}30)\, P(11{:}30)$. From the chart: $3 + 6 + 5 = 14$ active mosquitoes at 11:30.

Species	$p(420 \mid c)$	$P(c \mid 11{:}30)$	Product
Culex	0.00287	$3/14$	6.1×10^{-4}
Anopheles	0.00248	$6/14$	1.06×10^{-3}
Aedes	0.000027	$5/14$	9.6×10^{-6}

Resulting Probabilities

$P(\textit{Culex} \mid \ldots) = 36.5\%$ $\quad$ $P(\textit{Anopheles} \mid \ldots) = 63.0\%$ $\quad$ $P(\textit{Aedes} \mid \ldots) = 0.6\%$

Mosquito identification: More features

Assuming wingbeat and time are independent given the species:

$$P(c \mid 420\text{Hz}, 11{:}30) \propto P(c)\, p(420 \mid c)\, P(11{:}30 \mid c) \propto p(420 \mid c)\, P(c \mid 11{:}30)$$

Remark: Bayes Theorem replaces $P(c)\, P(11{:}30 \mid c)$ by $P(c \mid 11{:}30)\, P(11{:}30)$. From the chart: $3 + 6 + 5 = 14$ active mosquitoes at 11:30.

Species	$p(420 \mid c)$	$P(c \mid 11{:}30)$	Product
Culex	0.00287	$3/14$	6.1×10^{-4}
Anopheles	0.00248	$6/14$	1.06×10^{-3}
Aedes	0.000027	$5/14$	9.6×10^{-6}

Resulting Probabilities

$P(\textit{Culex} \mid \ldots) = 36.5\%$ $\quad$ $P(\textit{Anopheles} \mid \ldots) = 63.0\%$ $\quad$ $P(\textit{Aedes} \mid \ldots) = 0.6\%$

- *Anopheles stephensi* is the most likely culprit.

Remarks

At this point you should be wondering where was the training set in this example.

- You never saw the training set, you only saw the model: Wing beat frequencies distributions and mosquito activity diagram.
- The training set was used to build the wing beat frequencies laws and the distribution diagram. Once you have them, you don't need the training set anymore.

Important remark

You saw normalization constants pretty much everywhere in the calculations. You don't need them to classify new items. Unless you really want probabilities, you don't have to normalize your results. In most cases, you don't compute anything and the algorithm does it for you !

Outline

1 Introduction

2 Decision trees

3 Naive Bayes Classifiers

4 Comparison and summary

5 Bibliography

Decision trees vs Naive Bayes

	Decision tree	Naive Bayes
Model	rules (discriminative)	probabilities (generative)
Interpretability	very high (rules)	good (per-feature probabilities)
Training cost	one pass per tree level	one single pass
Prediction cost	depth of the tree	very fast
Feature scaling	not needed	not needed
Correlated features	handled	violates the hypothesis
Irrelevant features	ignored if not selected	little influence
Main risk	overfitting (high variance)	bias from independence
Typical use	decision support, tabular data	text, spam, fast baselines

In practice: scikit-learn

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.naive_bayes import GaussianNB
from sklearn.datasets import load_iris

X, y = load_iris(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
                                          random_state=0)

tree = DecisionTreeClassifier(criterion="entropy", max_depth=3)
tree.fit(X_tr, y_tr)
print(export_text(tree))                    # the tree as rules
print("Tree accuracy:", tree.score(X_te, y_te))

nb = GaussianNB().fit(X_tr, y_tr)
print("NB accuracy:", nb.score(X_te, y_te))
print(nb.predict_proba(X_te[:3]))           # posterior probabilities
```

Exercises

1. Using the full weather data, compute $IG(S, \text{Wind})$ by hand. Check that you get 0.048.
2. In the Rain branch of the weather tree, which attribute has the highest information gain?
3. Classify the day (Overcast, Hot, High, Weak) with Naive Bayes, without and with Laplace smoothing ($\alpha = 1$).
4. In the mosquito example, what is the most likely species for a wingbeat of 530 Hz (equal priors)?
5. Why does pruning never decrease the training error? What does it decrease?

Summary

- Supervised learning learns a model from labeled data; the test set is used once to measure generalisation.
- The bias–variance trade-off: choose the complexity that minimises the validation error.
- **Decision trees** split greedily on the attribute with the highest information gain (or Gini decrease); depth and pruning control overfitting.
- **Naive Bayes** applies Bayes theorem with conditional independence; use Laplace smoothing and log-probabilities in practice.

Next lecture (7.2)

Distance-based approaches (k-Nearest Neighbors) and validation criteria (confusion matrix, precision, recall, F1, cross-validation).

Bibliography

- L. Breiman, J. Friedman, R. Olshen and C. Stone. *Classification and Regression Trees*. Wadsworth, 1984.
- J. Cendrowska. PRISM: An algorithm for inducing modular rules. *International Journal of Man-Machine Studies*, 27(4):349–370, 1987.
- T. Hastie, R. Tibshirani and J. Friedman. *The Elements of Statistical Learning*, 2nd ed. Springer, 2009.
- T. Mitchell. *Machine Learning*. McGraw-Hill, 1997.
- J. R. Quinlan. Induction of decision trees. *Machine Learning*, 1(1):81–106, 1986.
- J. R. Quinlan. *C4.5: Programs for Machine Learning*. Morgan Kaufmann, 1993.
- I. H. Witten, E. Frank, M. A. Hall and C. J. Pal. *Data Mining: Practical Machine Learning Tools and Techniques*, 4th ed. Morgan Kaufmann, 2017.